

LA BATAILLE NAVALE



SOMMAIRE	1
INTRODUCTION.....	4
PRESENTATION DU JEU	5
Principe du jeu :	5
R��gles du jeu :	5
POINTS IMPORTANT DU PROGRAMME	6
Description des principales fonctionnalit��s :.....	6
VERSION 1 :.....	7
Architecture du code.....	7
Explication du code.....	8
VERSION 2 :.....	9
Am��liorations effectu��es.....	9
Explication du code.....	10
DIFFICULTES RENCONTREES	11
CONCLUSION	12

INTRODUCTION

Le projet que je vais pr  senter consiste en la cr  ation de plusieurs versions num  riques du jeu classique " La Bataille Navale", un jeu de strat  gie dans lequel deux joueurs essaient de couler les navires de l'adversaire. Le but de ce projet   tait de mettre en pratique des comp  tences en programmation sur Python, en conception de jeux et en gestion d'interface utilisateur. Ce rapport pr  sente les   tapes cl  s du d  veloppement du jeu, les principaux d  fis techniques rencontr  s, ainsi que les solutions apport  es pour y faire face.

PRESENTATION DU JEU

Principe du jeu :

Le jeu "Bataille Navale" est un jeu de strat  gie o   deux joueurs s'affrontent pour tenter de couler tous les navires de l'adversaire. Chaque joueur commence la partie avec une grille secr  te dans laquelle il place ses navires. Les navires sont repr  sent  s sur une grille de 10x10 cases, et chaque joueur ne conna  t pas l'emplacement des navires de son adversaire.

Au d  but du jeu, chaque joueur place ses navires sur sa propre grille de mani  re strat  gique. Une fois les navires positionn  s, les joueurs se relaient pour annoncer des tirs sur la grille de l'autre joueur. Un tir est annonc   sous la forme d'un couple de coordonn  es (lettre, chiffre), par exemple « A5 ». L'autre joueur r  pond alors si le tir tombe    l'eau, ou s'il touche un bateau. Si un bateau est touch  , il r  pond « touch   », sauf si ce dernier est enti  rement coul  , auquel cas il r  pond « touch  -coul   ». Le jeu se poursuit ainsi jusqu'   ce qu'un joueur ait r  ussi    couler tous les navires de son adversaire.

R  gles du jeu :

Les r  gles sont les suivantes :

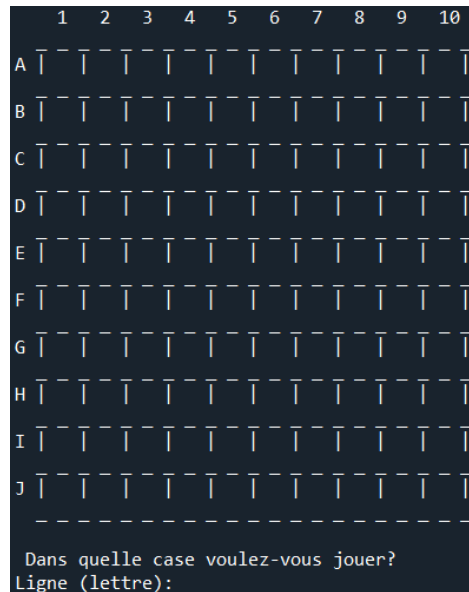
- **Placement des navires :** Chaque joueur place secr  tement ses navires sur une grille de 10x10. Les navires ne doivent pas se chevaucher et doivent   tre plac  s horizontalement ou verticalement. Il y a g  n  ralement cinq navires de diff  rentes tailles (par exemple : un porte-avions de 5 cases, un croiseur de 4 cases, un sous-marin de 3 cases, un torpilleur de 2 cases et une barque de 1 case).
- **Tours de jeu :**    chaque tour, un joueur annonce une case    l'adversaire (par exemple, "A5"). L'adversaire lui r  pond en fonction de l'emplacement de ses navires :
 - Si la case ne contient aucun navire, il r  pond « manqu   ».
 - Si la case touche un navire, il r  pond « touch   ».
 - Si le navire est enti  rement coul  , il r  pond « coul   ».
- **Fin de la partie :** Le jeu continue jusqu'   ce qu'un joueur ait coul   tous les navires de l'autre joueur. Le joueur qui y parvient en premier est d  clar   vainqueur.

POINTS IMPORTANTS DU PROGRAMME

Description des principales fonctionnalit  s :

Parmi les fonctionnalit  s principales du jeu "Bataille Navale", on trouve :

- **L'interface graphique :** Une interface simple permettant aux joueurs de placer leurs navires et de tirer sur la grille de l'adversaire.



- **Le placement des navires de l'ordinateur :** L'ordinateur place ses navires de mani  re al  atoire en v  rifiant que les positions choisies ne chevauchent pas d'autres navires.
- **Le m  canisme de tir :** Lorsque le joueur s  lectionne une case, le jeu v  rifie si un navire y est pr  sent et affiche le r  sultat ("touch  ", "coul  ", ou "manqu  ").
- **La gestion des erreurs :** Le jeu emp  che les actions invalides : il v  rifie que le tir est correct. Si le tir n'est pas correct, il redemande    l'utilisateur de jouer. Si le tir est correct, il retourne les coordonn  es de ce tir.
- **La d  tection de fin de partie :** Le jeu d  tecte automatiquement lorsque tous les navires d'un joueur sont coul  s, annon  ant ainsi la fin de la partie.

VERSION 1 :

Architecture du code

Le code est divis   en plusieurs fonctions principales :

- **PlacerBateauOrdi()** : Cette fonction g  re le placement al  atoire des navires de l'ordinateur sur la grille. Elle v  rifie les conditions de validit   du placement (pas de chevauchement et dans les limites de la grille).
- **InitialiserEcran() et afficheGrilleOrdi()** : Ces fonctions sont responsables de l'affichage de la grille vide de l'ordinateur et de la grille de jeu mise    jour apr  s chaque tir.
- **AfficheEcran()** : Cette fonction affiche la grille mise    jour avec les r  sultats des tirs (manqu  , touch  , ou coul  ).
- **JouerJoueur()** : Cette fonction g  re le tir du joueur, l'  valuation du r  sultat du tir (touch  , manqu  , coul  ), et la mise    jour de la grille de l'ordinateur et de l'  cran de jeu.

Le programme utilise un tableau NumPy (**Jeu**) pour repr  senter la grille de l'ordinateur, un tableau (**JeuInitial**) pour aussi repr  senter la grille de l'ordinateur mais qui ne la modifie pas au cours du jeu et un autre tableau (**Ecran**) pour afficher l'  tat du jeu au joueur.

Concernant le contenu de ces tableaux matriciels, j'ai choisi d'attribuer :

- 0 : eau
- 1 : barque
- 2 : torpilleur
- 3 : sous-marin
- 4 : torpilleur
- 5 : porte-avion

Explication du codeFonction PlacerBateauOrdi():

- **i** est un compteur repr  sentant la taille du bateau    placer. Il commence    1, ce qui signifie que le premier bateau sera de taille 1.    chaque it  ration de la boucle *while*, **i** est incr  ment   de 1, repr  sentant ainsi un bateau de taille plus grande.
- **a et b** : Ces variables repr  sentent les coordonn  es (lignes et colonnes) o   le bateau sera plac   sur la grille. Elles sont choisies de mani  re al  atoire dans la plage de 0    9, ce qui correspond aux indices des lignes et des colonnes de la grille 10x10.
- **b + i - 1 <= 9** : Cette condition v  rifie si la position de la fin du bateau est encore dans les limites de la grille. En effet, si le bateau est de taille **i** et qu'il doit   tre plac   horizontalement    partir de la colonne **b**, la derni  re case du bateau serait    la colonne **b + i - 1**. Cette condition garantit que cette derni  re case ne d  passe pas la limite de la grille (l'indice 9 pour la colonne).
- **if A[a][b + k] == 0** : Cette condition v  rifie si la case    la position $A[a][b + k]$ est vide (repr  sent  e par 0). Si la case est vide, le compteur **n** est incr  ment  .
- **n == i** : Si **n** atteint la taille du bateau **i**, cela signifie que toutes les cases n  cessaires    l'emplacement du bateau sont libres. Le bateau peut alors   tre plac   sur ces cases.
- **A[a][b + k] = i** : Si l'emplacement est valide, les cases sont remplies avec la taille du bateau (**i**), ce qui signifie que le bateau est plac   horizontalement sur la grille.
- **Placement   chou   (emplacement invalide)** : Si l'emplacement est invalide, soit parce que les cases sont occup  es, soit parce que le bateau d  passe la grille, le compteur **i** est d  cr  ment   de 1 (**i -= 1**). Cela permet de r  essayer de placer ce bateau    une autre position.

Fonction JouerJoueur():

- **Arguments :** Cette fonction prend en argument l'  cran et la grille du jeu, et met    jour l'  tat du jeu ainsi que l'  cran pour afficher les r  sultats apr  s chaque tir.
- **Attribution des valeurs :** La fonction attribut diff  rentes valeurs    la matrice Ecran en fonction du message re  u, ces valeurs serviront pour l'affichage dans la fonction *AfficheEcran()*. Cela permet de refl  ter l'  tat du jeu dans la matrice apr  s chaque action du joueur.

```

if message=="Manqu  ":
    E[i][j]=1
if message=="Touch  ":
    E[i][j]=2
if message=="Coul  ":
    for n in range(10):
        for m in range(10):
            if JeuInitial[n][m]==elmt:
                E[n][m]=3

```

```

if E[n][m]==0:
    B.append(" ")
elif E[n][m]==1:
    B.append("*")
elif E[n][m]==2:
    B.append("+")
else:
    #E[n][m]==3:
    B.append("X")

```

- **Jeu[i][j] = 100 :** Cette valeur est utilis  e pour marquer que cette case a d  j   t   cibl  e. Cela   vite ainsi de jouer plusieurs fois sur la m  me case et garantit que le joueur ne puisse pas tenter un tir sur une case d  j attaqu  e.

VERSION 2 :Am  liorations effectu  es

Dans cette nouvelle version du jeu "Bataille Navale", plusieurs modifications et am  liorations ont   t   apport  es par rapport    la premi  re version.

Premi  rement, j'ai impl  ment   une fonction *AfficheGrilleJoueur()* qui permet d'afficher la grille du joueur, o   les bateaux sont repr  sent  s par des lettres (par exemple, "B" pour Barque, "T" pour Torpilleur, etc.).

Puis une fonction *afficheEcransDouble()* qui permet d'afficher simultan  ment les grilles du joueur et de l'ordinateur. Cela permet au joueur de voir les informations sur sa propre grille ainsi que sur celle de l'ordinateur, tout en masquant certains d  tails de l'ordinateur (par exemple, les positions des bateaux cach  s).

	1	2	3	4	5	6	7	8	9	10		1	2	3	4	5	6	7	8	9	10	
A											+	A					B					
B				*								B										
C												C								*		
D												D										*
E					*							E				*						
F												F					T	T				
G												G			*							
H						+						H					S	+	S			
I												I					C	C	C	C		
J												J					P	P	P	P	P	
	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----		-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Et enfin, le jeu alterne entre les tours du joueur et de l'ordinateur : le joueur choisit ses tirs manuellement, tandis que l'ordinateur tire al  atoirement sur la grille du joueur.

Explication du code

Fonction PlacerBateauJoueur():

Dans cette fonction, j'ai d  cid   de refaire les m  mes v  rifications que dans la fonction *PlacerBateauOrdi()* afin de r  duire le risque d'erreur. Cela permet de garantir que le placement des bateaux du joueur respecte les m  mes r  gles de validit   que celles appliqu  es pour l'ordinateur, assurant ainsi une logique coh  rente et fonctionnelle entre les deux versions du programme.

Fonction Jouer():

Dans cette version, j'ai d  cid   de mettre en argument les diff  rents navires car il y a maintenant deux joueurs. Chaque joueur poss  dant ses propres navires, cette fonction m'a permis de cibler sp  cifiquement les navires sur lesquels je voulais effectuer mon programme. Cette approche me permettait un suivi individuel des navires pour chaque joueur.

DIFFICULTES RENCONTREES

1er probl  me :

Le plus gros probl  me que j'ai rencontr     tait de mettre des **X** aux endroits o   il y avait d  j   des **+**. En effet, au d  but, pour savoir si mes codes fonctionnaient, j'affichais la grille de l'ordinateur avec les bateaux que ce dernier a plac  s. De ce fait, quand je jouais je retirais les bateaux touch  s (en mettant des 0    la matrice repr  sentatrice). Ce qui fait qu'en rempla  ant par des 0 les bateaux touch  s, je perdais leurs coordonn  es et donc je n'arrivais pas    remplacer les **+** par des **X**.

Cependant, comme dans le jeu on ne veut pas afficher la grille de jeu de l'ordinateur, j'ai tout simplement retir   les lignes de code o   la matrice prenait des valeurs 0 aux endroits touch  s, ainsi je pouvais conserver les coordonn  es de mes anciens bateaux.

2e probl  me :

J'ai rencontr   des probl  mes avec le fait de jouer sur une case d  j   jou  e. Comme je ne changeais pas les valeurs de la matrice repr  sentative de la grille de l'ordinateur, mon programme ne pouvait pas savoir si j'avais d  j   jou      tel endroit ou pas. De ce fait, j'ai d  cid   de cr  er une copie de cette matrice : "**Jeulnitial**". Ainsi, je pouvais changer les valeurs de ma matrice initiale (en mettant des 100 aux endroits jou  s) et je pouvais toujours r  soudre mon premier probl  me (car la matrice **Jeulnitial** reste inchang  e et contient les coordonn  es de tous les bateaux).

CONCLUSION

Ce projet a   t   une excellente occasion de mettre en pratique mes comp  tences en programmation, notamment en Python, tout en concevant un jeu interactif. Bien que des d  fis techniques se soient pr  sent  s, les solutions apport  es ont permis d'am  liorer la version initiale et d'offrir une exp  rience de jeu plus fluide et plus engageante. Les am  liorations dans la version 2, telles que l'ajout de l'interface graphique et l'alternance entre les tours du joueur et de l'ordinateur, enrichissent l'exp  rience de jeu. Ce projet a donc   t   une excellente opportunit   pour renforcer mes comp  tences en programmation Python.